Exploring NeoRL: Critic-Regularized Regression with Past

Haotian Chu

Abstract

Offline reinforcement learning (RL), also known as batch RL, offers the prospect of policy optimization from large pre-recorded datasets without online environment interaction. Based on NeoRL benchmark (Qin et al., 2021), we apply a novel offline RL algorithm to learn policies from stock data using a form of **c**ritic-**r**egularized **r**egression, named CRR (Wang et al., 2020). Considering the finance setting, we provide the agent not only the current observation but also historical data (thus it is called "**CRR with Past**"), and find out that this feature is quite essential in improving the performance. What's more, we compare several variants of CRR and discuss the choice of different weight functions.

1. Introduction

In quantitative finance, stock trading is essentially making dynamic decisions, namely to decide where to trade, at what price, and what quantity, over a highly stochastic and complex stock market. With the development of technology, traders began to cast interest in automatic methods when dealing with finance problems. Offline reinforcement learning (RL) is one of those popular options.

In offline RL, also known as Batch RL, the environment is no longer available during training, and the agent only relies on a static dataset D. The data can be gathered by some unknown behavioral policy (or a mixture of policies) π_μ. Because the data is static and the policy has no access to the environment during training, the agent is faced with the problem of extrapolation error (Kidambi et al., 2020), as well as losing its ability to correct its estimation inaccuracies.

The naive application of off-policy RL algorithms with function approximation to the offline setting has often failed, and a shared conclusion is that failure comes from overly optimistic Q-estimates, as well as inappropriate extrapolation beyond the observed data.

In this paper, we choose **c**ritic-**r**egularized **r**egression (Wang et al., 2020) algorithm to learn policies from data. CRR essentially reduces offline policy optimization to a form of value-filtered regression that requires minimal algorithmic changes to standard actor-critic methods.

In the experiment, we rely heavily on the user interface and results of NeoRL (Qin et al., 2021), a suite of near real-world benchmarks for offline RL. Beyond the finance setting, NeoRL includes robotics, industrial control, and city management tasks with real-world nature. Currently, NeoRL provides different sizes from low to high amounts, and different quality of data collected from corresponding simulators, and benchmark recently proposed model-free and model-based offline RL methods as a reference. The online and offline evaluations are both performed for selecting the hyper-parameters for each algorithm.

After exploring the finance dataset of NeoRL, we find that the observation vector is merely a snapshot of daily stock market. Although it makes sense in the setting of reinforcement learning, it seems to contradict to the logic of finance, where traders usually make the decision of purchase (selling) after analyzing the historical data for a long time.

As a result, we derive **"CRR with Past"**. considering the sequential logic of stock data, we concatenate the past t days' prices information to the observation vector, providing the agent the observation within a period of time instead of a simple snapshot.

What's more, our experiments discuss several variants of CRR model and show that they outperforms several state-of-the-art offline RL algorithms by a significant margin.

2. Momentum Indicators

Considering the finance setting of the environment, we will introduce some economic analyzing techniques to help make investment decisions(Liu et al., 2020).

MACD. Developed by Gerald Appel, the Moving Average Convergence Divergence oscillator (MACD) is one of the simplest and most effective momentum indicators available. The MACD turns two trend-following indicators into a momentum oscillator by subtracting the longer moving average from the shorter moving average. Because the MACD is unbounded, it is not particularly useful for identifying over-

bought and oversold levels.

RSI. Relative Strength Index (RSI) is another momentum indicator used in technical analysis that measures the magnitude of recent price changes to evaluate overbought or oversold conditions in the price of a stock or other asset. The RSI is displayed as an oscillator (a line graph that moves between two extremes) and can have a reading from 0 to 100. The indicator was originally developed by J. Welles Wilder Jr. and introduced in his seminal 1978 book, "New Concepts in Technical Trading Systems."

CCI. The Commodity Channel Index (CCI) is a momentum-based oscillator used to help determine when an investment vehicle is reaching a condition of being overbought or oversold. Developed by Donald Lambert, this technical indicator assesses price trend direction and strength, allowing traders to determine if they want to enter or exit a trade, refrain from taking a trade, or add to an existing position. In this way, the indicator can be used to provide trade signals when it acts in a certain way.

All the three indicators are included in the observation vector as shown in Fig. 1.

Figure 1. The Observation Vector is a concatenation of the current balance, prices, MACD, RSI and CCI of the total 30 stocks.

3. Critic Regularized Regression

Critic Regularized Regression (CRR), is a simple, yet effective, method for offline RL.

3.1. Policy Learning with CRR

Firstly, consider the policy evaluation in the setting of offline RL. Suppose we are given a fixed dataset $\mathcal{B}$ containing either individual transition tuples (s_t, a_t, r_t, s_{t+1}). An important step in many off-policy learning approaches is off-policy policy evaluation. This often involves learning an approximation to the Q-function minimizing a temporal difference loss similar to the following

$$\mathbb{E}_{\mathcal{B}}\Big[D\big(Q_\theta(s_t, a_t), (r_t + \gamma \mathbb{E}_{a\sim\pi(s_{t+1})} Q_{\theta'}(s_{t+1}, a))\big)\Big] \quad (1)$$

where the expectation over data is approximated by sampling $(s_t, a_t, r_t, s_{t+1}) \sim \mathcal{B}$ and D is some divergence measure; measuring discrepancy between the current estimate of the action-value $Q_\theta(s_t, a_t)$ (for policy π) and the td-update, based on a target network (Mnih et al., 2015). We note that we make use of a distributional Q-function as in Barth-Maron et al. (Barth-Maron et al., 2018), instead of using a squared error for D.

As discussed in recent work (Fujimoto et al., 2019), due to lack of environment interaction while evaluating the value of the next state s_{t+1} by $\mathbb{E}_{a\sim\pi(s_{t+1})} Q_{\theta'}(s_{t+1}, a)$, this loss may be problematic. If trained with standard RL, a parametric $\pi(\cdot|s_{t+1})$ is likely to extrapolate beyond the training data and to propose actions that are not contained in $\mathcal{B}$. For these actions Q will not have been trained and may produce bad estimates.

To mitigate the problem, we should avoid evaluating Q for $(s, a) \notin \mathcal{B}$. Thus, we aim to train π by discouraging it from taking actions that are outside the training distribution. Such a requirement would be hard to achieve with standard policy gradients (Silver et al., 2014). We thus change our objective to match the state-action mapping contained in the training data-but filtered by the Q-function. Specifically, we optimize

$$\arg\max_{\pi} \mathbb{E}_{(s,a)\sim\mathcal{B}}\Big[f(Q_\theta, \pi, s, a) \log \pi(a|s)\Big] \quad (2)$$

where f is a non-negative, scalar, function whose value is monotonically increasing in Q_θ. Fundamentally, Eq. (2) tries to copy actions that exist in the data, thereby restricting π. Further properties of the learning objective can be controlled through different choices of f, for example when $f := 1$, Eq. (2) is equivalent to Behavioral Cloning (BC). The success of BC is, however, highly dependent on the quality of the dataset $\mathcal{B}$. When $\mathcal{B}$ does not contain enough transitions generated by a policy performing well on the task, or the fraction of poor data is too large, then BC is likely to fail.

Algorithm 1 Critic Regularized Regression

Input: Dataset $\mathcal{B}$, critic net Q_θ, actor net π_ϕ, target actor and critic nets: $\pi_{\phi'}$, $Q_{\theta'}$, function f
for $n_{updates}$ **do**
  Sample $(s_t^i, a_t^i, r_t^i, s_{t+1}^i)_{i=1}^b$ from $\mathcal{B}$.
  Update actor (policy) with gradient:
    $\nabla_\phi - \frac{1}{b}\sum_i \log \pi_\phi(a_t^i|s_t^i) f(Q_\theta, \pi_\phi, s_t^i, a_t^i)$
  Update critic with gradient:
  $\nabla_\theta \frac{1}{b}\sum_i D[Q_\theta(s_t^i, a_t^i), (r_t^i + \gamma \mathbb{E}_{a\sim\pi_{\theta'}} Q_{\theta'}(s_{t+1}^i, a))]$
  Update the target actor/critic nets every N steps by copying parameters: $\theta' \leftarrow \theta, \phi' \leftarrow \phi$.
end for

3.2. Advantage Estimators

Provided Q is sufficiently accurate for $(s, a) \in \mathcal{B}$ (e.g. learned using Eq. (1)), we can consider additional choices of f that enable off-policy learning to overcome this problem:

$$f := \mathbb{1}[\hat{A}_\theta(s, a) > 0] \quad (3)$$

$$f := \exp\big(\hat{A}_\theta(s, a)/\beta\big) \quad (4)$$

where $\beta > 0$ is a hyper-parameter, $\mathbb{1}$ the indicator function, and $\hat{A}_\theta(s,a)$ is an estimated advantage function. Intuitively, Eq. (3) entails BC on a filtered dataset where the filtering increases the average quality of the actions we learn from. The filter is defined in terms of the value of the current policy. For each f, we should further consider different advantage estimators:

$$\hat{A}_{mean}(s_t,a_t) = Q_\theta(s_t,a_t) - \frac{1}{m}\sum_{j=1}^{m} Q_\theta(s_t,a^j) \quad (5)$$

$$\hat{A}_{max}(s_t,a_t) = Q_\theta(s_t,a_t) - \max_{j=1}^{m} Q_\theta(s_t,a^j) \quad (6)$$

In the beginning, we intended to simply consider $\hat{A}_{mean}$, but according to the original paper of CRR (Wang et al., 2020), for small m, $\hat{A}_{mean}$ may overestimate the advantage due to stochasticity. We therefore also consider $\hat{A}_{max}$, a pessimistic estimate of the advantage.

As we have two choices of weight functions and two different types of advantage estimators, we can derive four variants of CRR algorithm. In Table 1, we have made the comparison of four variants.

Table 1. The online evaluation results trained on **raw data**. Policies are measured by their episode return and averaged over 100 episodes.

VARIANTS	HIGH-100	MEDIUM-100	LOW-100
MEAN-EXP	356	315	328
MEAN-BINARY	229	251	174
MAX-EXP	**380**	**332**	335
MAX-BINARY	308	170	**438**

"MEAN" corresponds to Eq. (5) and "MAX" is Eq. (6), while "BINARY" corresponds to Eq. (3) and "EXP" corresponds to Eq. (4). "HIGH-100" represents the dataset "finance-high-100" of NeoRL (Qin et al., 2021).

From the table, we can roughly reach a conlcusion that the pessimistic estimator $\hat{A}_{max}$ performs better than $\hat{A}_{mean}$, but the difference of weight functions is still subtle. In the following section, we will gain insight into Eq. (4).

3.3. Exponential Weighting

We can gain insight into Eq. (4) by realizing that it approximately implements a regularized policy improvement step in a policy iteration scheme. Consider

$$\mathcal{L}(q,\mu_\mathcal{B}) = \mathbb{E}_{s\sim\mathcal{B}}\Big[\mathbb{E}_q[Q_\theta(s,a)] + \beta \mathrm{KL}[q(\cdot|s),\mu_\mathcal{B}(\cdot|s)]\Big] \quad (7)$$

where $\mu_\mathcal{B}$ is the policy that generated the data, and $q(a|s) \propto \exp(\hat{A}_\theta(s,a)/\beta)$. By minimizing the cross-entropy $H[q(\cdot|s),\pi_\phi(\cdot|s)]$, we can project this distribution back onto the parametric policy π_ϕ. In Fig. 2, we plot the average reward of different β. When trained on medium quality, the best performance take place when β equals to 4. It should be noted that, as $\beta \to \infty$, this objective becomes behavior cloning (BC).

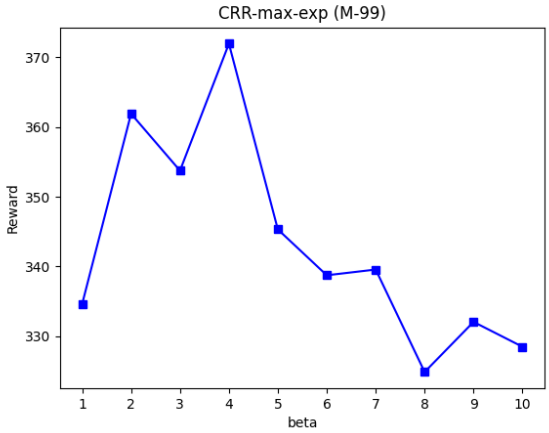

Figure 2. Reward of different β.

4. The effect of past information

In the setting of reinforce learning, the input of the model is usually a snapshot of the environment, which is commonly accepted in various RL frameworks. However, in some downstream tasks, "memory" (historical data) is important during decision making, and stock trading is a definitely one of them.

In reality, traders usually make the decision of purchase (selling) after analyzing the historical data for a long time. Some trading strategies even rely on historical data to conclude the market trend.

As a result, instead of simply feeding raw data into CRR model, we concatenate the past t days' prices information to the observation vector (Fig. 3), enabling the agent to take the past into consideration while making decision.

Figure 3. The information of past t days' prices is concatenated to the observation vector.

To test the effect of historical information, we compare the performance when models are trained on raw data and data

with past information. And the result is plotted as follows. (The remains are in the Appendix.)

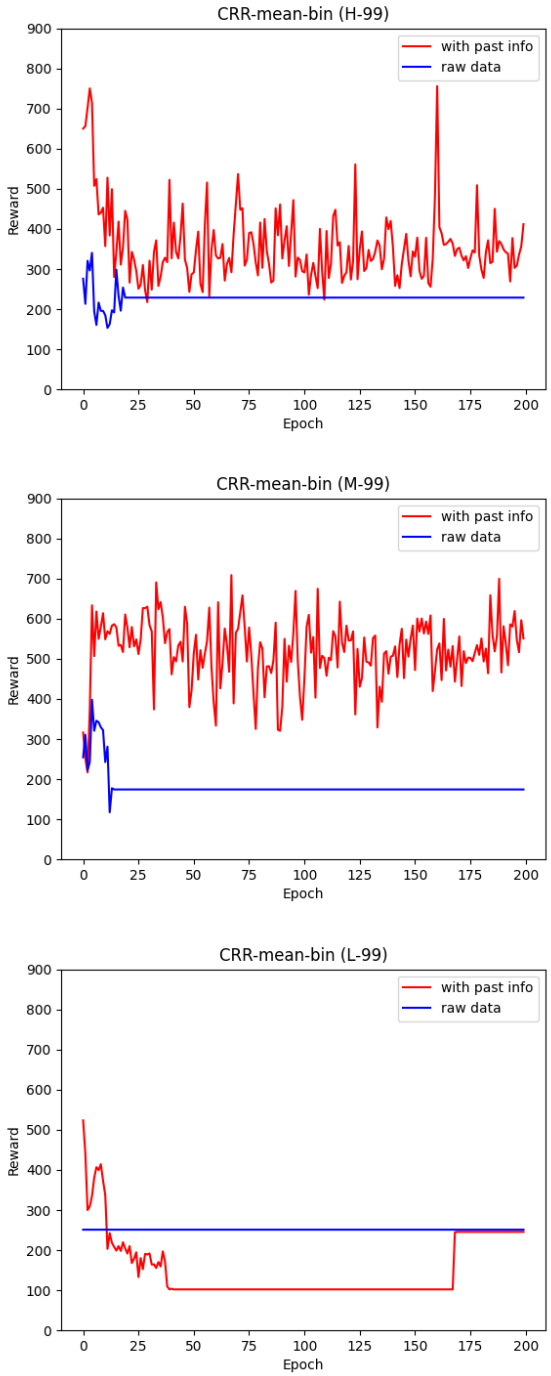

Figure 4. Reward of CRR in the first 200 epochs. In this figure, weight function is chosen according to Eq. (3) and Eq. (5). "H-99" represents "finance-high-100" dataset of NeoRL (Qin et al., 2021).

The figure shows that CRR model trained on data with past information outperforms that trained on raw data, especially when data is of medium and high quality.

To generalize the conclusion, we also make the comparison based on Behavioral Cloning (BC) algorithm, which is a baseline of learning method in the NeoRL benchmark.

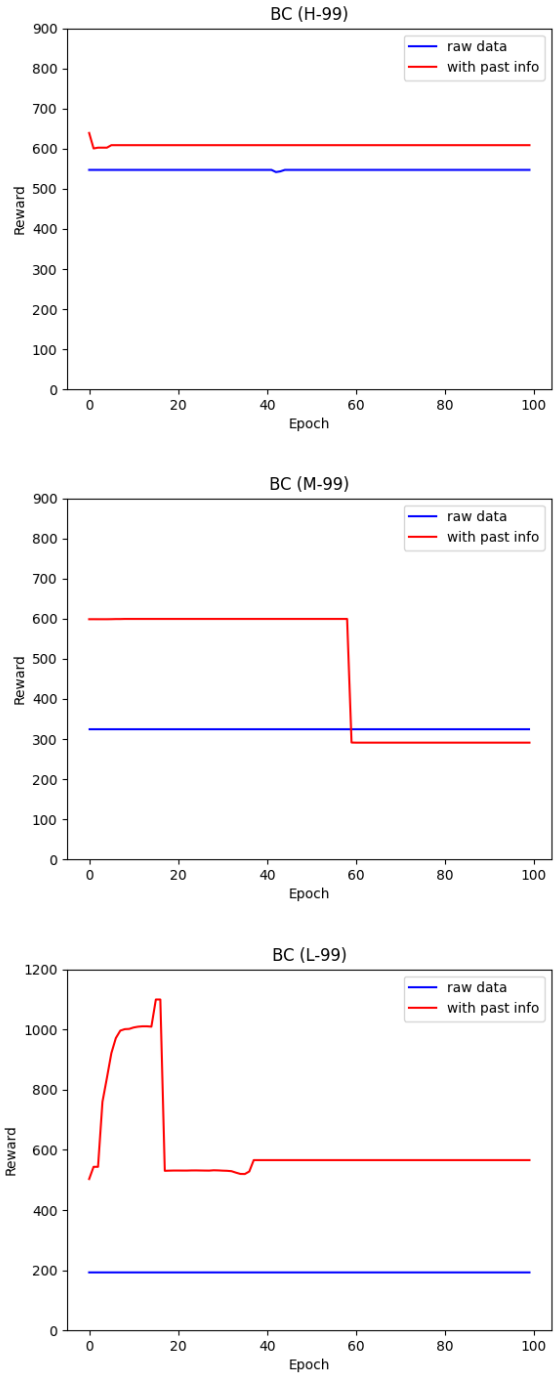

Figure 5. Reward of BC in the first 200 epochs.

We can conclude from the figure that training on data that

contains historical information helps BC improves its performance too. It should be noticed that, learned from high-quality processed data, BC can achieve performance similar to the expert policy according to NeoRL benchmark (Qin et al., 2021).

Then, in order to test the effect of past information on CRR variants, we train four variants using processed data (Table 2).

Table 2. The online evaluation results trained on **processed data**. The number in the parenthesis stands for the difference between evaluation results trained on raw data.

VARIANTS	HIGH-100	MEDIUM-100	LOW-100
MEAN-EXP	250 (-156)	**540** (+225)	**318** (-10)
MEAN-BINARY	353 (+124)	533 (+282)	194 (+20)
MAX-EXP	286 (-94)	509 (+177)	301 (-34)
MAX-BINARY	**378** (+70)	458 (+288)	176 (-262)

From the comparison with Table 1, we can easily find out that the performance of optimistic estimator $\hat{A}_{mean}$ grows significantly after we provide the agent with historical information, which implies that historical information might lower the probability of overestimation.

It should be noted that, interestingly, the effect of historical data is tremendously marvelous on dataset of medium quality. One possible explanation might be that, agent can already learn mature strategy from high-quality raw dataset, therefore the additional historical information can hardly further improve the result.

At last, we plot the average reward of different β using processed data (Fig. 6).

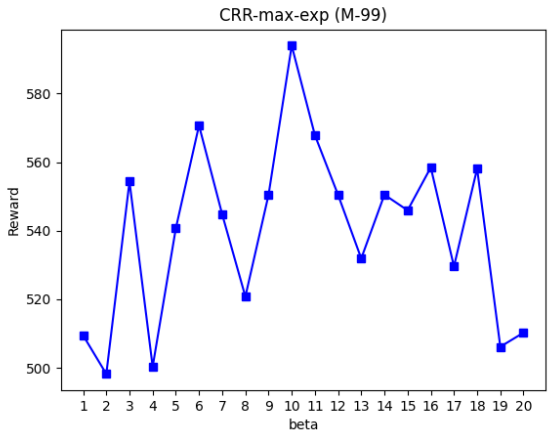

Figure 6. Reward of different β when trained on processed data.

Different from Fig. 2, when trained on data with historical information, the reduction of performance appears when $\beta > 10$. (In previous experiment, the performance declines when $\beta > 4$.)

5. Conclusion

From several simple experiments and comparison with NeoRL benchmark, we have depicted a rough picture of CRR model as well as its variants, and have provided a comprehensive discussion about the effect of historical data.

When models are trained on raw data, we find on that the pessimistic estimator $\hat{A}_{max}$ performs better than $\hat{A}_{mean}$, while the difference between binary and exponential weight function is still subtle (Table 1). Considering the exponential weight function, the best performance take place when β equals to 4 (Fig. 2, trained on raw data of medium quality for example), which does make sense because the model becomes behavior cloning (BC) as $\beta \to \infty$, which is a baseline learning method of NeoRL benchmark.

Although it is generally accepted that the input of RL model is a snapshot of current environment, we think that it contradicts to the logic of stock trading. Therefore, considering the finance setting, we concatenate the past t days' prices information to the observation vector, and find that CRR model trained on data with past information outperforms that trained on raw data, especially when data is of medium and high quality (Fig. 4). To generalize the effect of historical information, we also test BC algorithm, and find that training on processed data helps BC improves its performance too.

We also train four variants using processed data to futher explore the effect of past information (Table 2), and find out that the performance of optimistic estimator $\hat{A}_{mean}$ grows significantly after the data are processed, which implies that historical information might lower the probability of overestimation. From the comparison with previous experiment, we surprisingly discover that the effect of historical data is unbelievably marvelous on dataset of medium quality. We think it might because agent can already learn mature strategy from high-quality raw dataset, therefore the additional historical information can hardly further improve the result. What's more, when trained on data with historical information, the reduction of performance appears when $\beta > 10$ (Fig. 6).

On the other hand, there exist some weaknesses to this work and further exploration can be done in the future. For example, the effect of historical data can be tested on algorithms of more diversity (CQL maybe, or some model-based methods), because BC is actually a special form of CRR as well. Also, although the idea of "CRR with past" creates great performance enhancement in the finance setting, whether

this improvement can be generalized to other downstream tasks remains to be explored.

References

Barth-Maron, G., Hoffman, M. W., Budden, D., Dabney, W., Horgan, D., Tb, D., Muldal, A., Heess, N., and Lillicrap, T. Distributed distributional deterministic policy gradients. *arXiv preprint arXiv:1804.08617*, 2018.

Fujimoto, S., Meger, D., and Precup, D. Off-policy deep reinforcement learning without exploration. In *International Conference on Machine Learning*, pp. 2052–2062. PMLR, 2019.

Kidambi, R., Rajeswaran, A., Netrapalli, P., and Joachims, T. Morel: Model-based offline reinforcement learning. *Advances in neural information processing systems*, 33: 21810–21823, 2020.

Liu, X.-Y., Yang, H., Chen, Q., Zhang, R., Yang, L., Xiao, B., and Wang, C. D. Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance, 2020.

Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533, 2015.

Qin, R., Gao, S., Zhang, X., Xu, Z., Huang, S., Li, Z., Zhang, W., and Yu, Y. Neorl: A near real-world benchmark for offline reinforcement learning, 2021.

Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., and Riedmiller, M. Deterministic policy gradient algorithms. In *International conference on machine learning*, pp. 387–395. PMLR, 2014.

Wang, Z., Novikov, A., Zolna, K., Springenberg, J. T., Reed, S., Shahriari, B., Siegel, N., Merel, J., Gulcehre, C., Heess, N., and de Freitas, N. Critic regularized regression, 2020.

A. The effect of past information

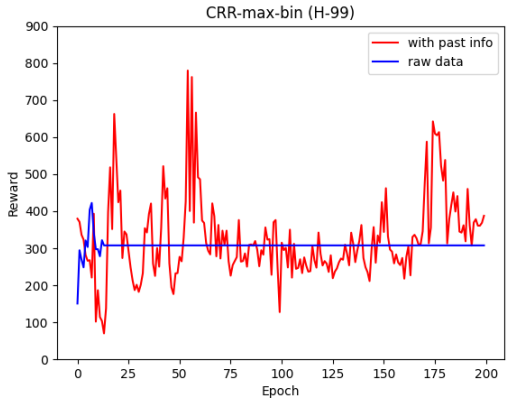

Figure 7. CRR-max-bin trained on "finance-high-100".

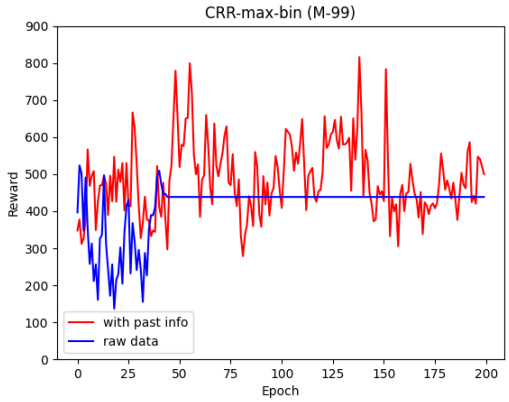

Figure 8. CRR-max-bin trained on "finance-medium-100".

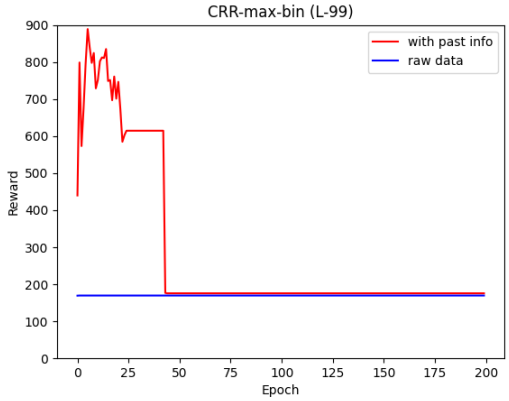

Figure 9. CRR-max-bin trained on "finance-low-100".

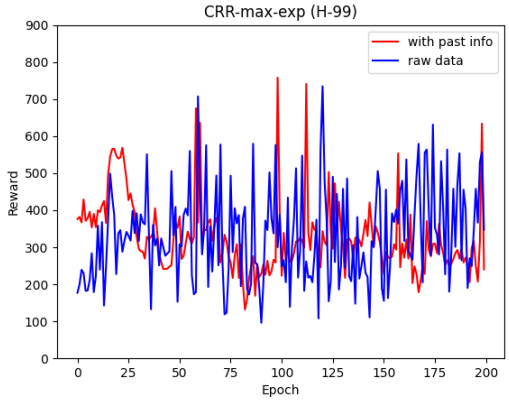

Figure 10. CRR-max-exp trained on "finance-high-100".

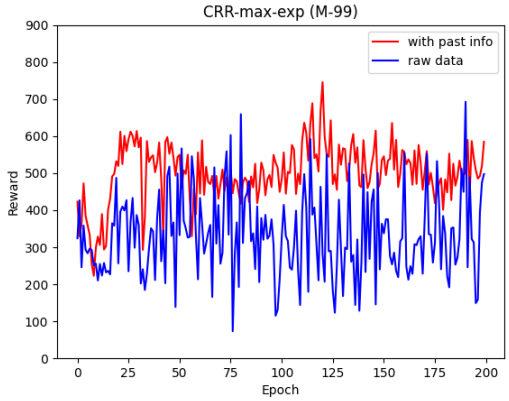

Figure 11. CRR-max-exp trained on "finance-medium-100".

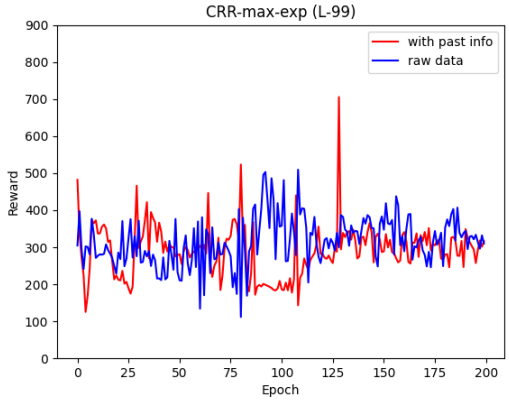

Figure 12. CRR-max-exp trained on "finance-low-100".

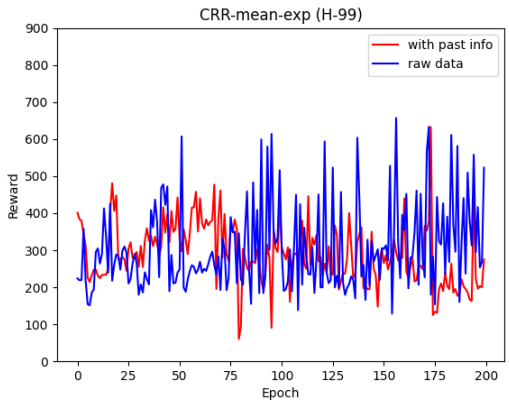

Figure 13. CRR-mean-exp trained on "finance-high-100".

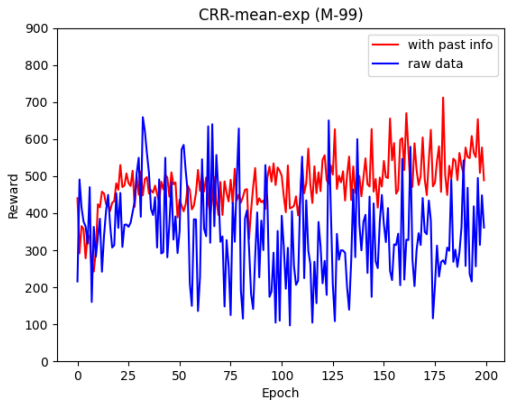

Figure 14. CRR-mean-exp trained on "finance-medium-100".

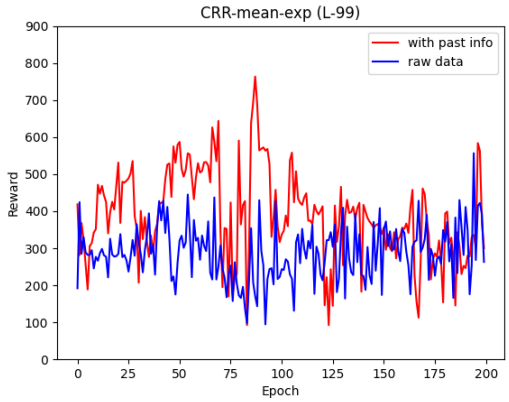

Figure 15. CRR-mean-exp trained on "finance-low-100".